

WikiRMI — scenariusz prezentacji (10 minut)

Zespół: Maciej Wawer, Krystian Wasił, Marek Więcaszek · **Temat 9** — system typu wiki · **Nacisk:** wielowątkowość (4 mechanizmy + RMI) · ~7 slajdów

CZĘŚĆ 1 — Maciej Wawer · ~3 min · Architektura rozproszona (RMI) i skąd biorą się wątki

Slajd 1 — tytuł (20 s)

„Dzień dobry. Przedstawiamy WikiRMI — rozproszony system typu wiki, temat 9. Administrator tworzy szkielety stron i konta, a użytkownicy je modyfikują. Komunikacja działa przez RMI, a sercem projektu jest bezpieczna współbieżność.”

Slajd 2 — schemat Klient–Rejestr RMI–Serwer (~1:30)

- Trzy warstwy: common (kontrakt), server, client.
- Interfejs zdalny `WikiService` extends `Remote`, metody throws `RemoteException` — `WikiService.java:15`.
- Serwer eksportuje `WikiServiceImpl` extends `UnicastRemoteObject` i rejestruje:
`Naming.rebind("//localhost:1099/WikiService")` — `WikiServer.java:35-36`.
- Klient pobiera stub: `Naming.lookup(...)` i woła metody jak lokalne — `WikiClientController.java:35`.
- RMI działa też w drugą stronę — callbacki serwer→klient (powiadomienia na żywo).

Most do wielowątkowości — KLUCZOWE zdanie:

„I tu pojawia się problem współbieżności: RMI obsługuje każde wywołanie klienta w osobnym wątku po stronie serwera. Gdy kilku użytkowników działa jednocześnie, wiele wątków serwera sięga po te same dane — bez synchronizacji powstałby stan wyścigu. Jak to rozwiązaliśmy — opowie Krystian.”

CZĘŚĆ 2 — Krystian Wasił · ~4 min · Wielowątkowość — serce projektu (4 mechanizmy)

Slajd 3 — sekcja krytyczna (~50 s)

- Problem: dwóch użytkowników edytuje tę samą stronę naraz → utrata zapisu (lost update) lub odczyt niespójnej strony.

„Cały rdzeń współbieżności umieściliśmy w jednym pliku — `WikiStore.java` — i celowo użyliśmy w nim czterech klasycznych mechanizmów, każdy w osobnej, opisanej sekcji.”

Slajd 4 — cztery mechanizmy (najważniejszy slajd, ~2:30)

- **1. BLOKADY — ReentrantReadWriteLock** (`Page.java:23`, `WikiStore.java:266`): każda strona ma własną blokadę (drobnoziarniste); `readLock()` = wielu czytelników, `writeLock()` = jeden piszący.
- Sekcja krytyczna „sprawdź-i-ustaw” blokady edycji (`WikiStore.java:321`): z N wątków DOKŁADNIE JEDEN zakłada dzierżawę, reszta dostaje `PageLockedException`. To nasza ochrona przed race condition.
- **2. MONITORY — synchronized** (`WikiStore.java:133`): wspólny licznik zapisów; bez `synchronized` „licznik++” gubiłby zliczenia (odczyt-dodaj-zapis nie jest atomowy).
- **3. SEMAFORY — Semaphore** (`WikiStore.java:51`, `67`): limit jednoczesnych klientów; logowanie `tryAcquire()`, wylogowanie `release()`.
- **4. WĄTKI — Thread daemon + pula** (`WikiStore.java:153`, `NotificationService.java:29`): wątek daemon zwalnia przeterminowane blokady; pula wątków (`ExecutorService`) wysyła powiadomienia poza wątkiem edycji.

Dwa mocne argumenty na koniec (~40 s):

- **Brak zakleszczeń:** każda operacja trzyma najwyżej jedną blokadę strony naraz → brak cyklu oczekiwań (`WikiStore.java:33-35`).
- **Brak lost update:** dodatkowo optymistyczna kontrola wersji — zapis sprawdza numer wersji.

„Pokażemy teraz, że to naprawdę działa — Marek.”

CZĘŚĆ 3 — Marek Więcaszek · ~3 min · Demo na żywo + testy współbieżności + dane

Slajd 5 — DEMO NA ŻYWO (~1:30) · dwa okna klienta już zalogowane

- W obu oknach zaznacz „Strona główna” → w kliencie 1 kliknij Edytuj.
- W kliencie 2 kliknij Edytuj tę samą stronę → komunikat „Strona jest aktualnie edytowana przez 'admin'...” → race condition zablokowany na żywo.
- W kliencie 1 zmień tekst i Zapisz → lista w kliencie 2 odświeża się automatycznie (callback RMI).

„Drugi klient nie mógł wejść w edycję — blokada zadziałała, a powiadomienie przyszło przez RMI.”

Slajd 6 — testy współbieżności (~1:00)

- ConcurrencyTest: 10 wątków próbuje naraz → 1 zdobywa blokadę, 9 odrzuconych (dowód braku wyścigu).
- LostUpdateTest: 50 edytorów → wersja = 50, nic nie zgubione.
- Łącznie 17/17 testów + test integracyjny po RMI (wyścig „przez sieć”).
- Dane w wiki.json (zapis atomowy) — po restarcie serwera stan jest odtwarzany; limit klientów pilnuje semafor.

Slajd 7 — podsumowanie (~30 s)

„Podsumowując: kompletny system rozproszony RMI z poprawną, drobnoziarnistą i wolną od zakleszczeń współbieżnością, pokazaną na czterech mechanizmach. Kod i dokumentacja są w repozytorium. Dziękujemy — zapraszamy do pytań.”

Wskazówki na obronę — każdy zna swój kawałek

- **Maciej** — RMI: czemu extends Remote / UnicastRemoteObject, różnica Naming.rebind vs rejestr, czemu callbacki.
- **Krystian** — różnica monitor vs blokada vs semafor; czemu ReadWriteLock zamiast synchronized (przepustowość odczytów); argument o braku deadlocka.
- **Marek** — jak testy dowodzą braku wyścigu (CountDownLatch, 10 wątków); jak działa restart i limit klientów.
- Bądźcie gotowi zmienić kod na żywo (np. limit semafora = 1 i pokazać odmowę logowania, albo skrócić czas blokady).